

MANAGEMENT BRIEFING

FinOps: Visibility & Governance

Seeing cloud spend — and controlling it

Presented to
Executive Leadership

Date

Spend you can see



THE PROBLEM WE FACE

Cloud spend is dynamic, distributed and opaque



Costs move daily

Spend is dynamic and distributed across teams, services and regions.



Budgeting models don't fit

Traditional IT budgeting assumes fixed, predictable costs. Cloud is neither.



Engineers spend money

Technical decisions made daily carry direct, immediate financial consequences.



Bills arrive without context

Monthly invoices show what was spent — rarely who spent it, or why.

“When costs rise and no one can explain who owns the change or what caused it, visibility has failed — even if a dashboard is showing numbers.”

The result

Sticker shock, finger-pointing and reactive cost-cutting instead of strategic optimization.

DEFINITION

FinOps = Finance + DevOps

An operational framework and cultural practice for maximizing the business value of cloud investment. Not a cost-cutting programme — a way to make the financial impact of technical choices visible to everyone.



01

Inform

Build visibility. Understand who is spending what, where, and why.



02

Optimize

Act on the insight. Rightsize, eliminate waste, manage commitments.



03

Operate

Embed cost awareness into daily engineering and business process.

A continuous cycle, not a sequence of projects.

“Without visibility, nothing else in the practice works.”

WHY VISIBILITY FIRST

You cannot optimize what you cannot see

Four questions the organization must be able to answer before any meaningful optimization begins.

Which teams, products or features drive cloud spend?

Accountability

Is spend growing with the business — or faster than it?

Efficiency signal

Which resources are paid for but sitting unused?

Waste identification

Are costs allocated accurately enough to drive ownership?

Behaviour change

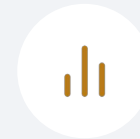
The risk of skipping visibility: optimization becomes reactive and superficial — blanket cost-cutting instead of targeted action against real waste.

What “good visibility” looks like



Cost allocation

Breakdown by team, product, environment or business unit. Depends entirely on a consistent tagging strategy.



Granular reporting

Breakdown by service, resource type and time period — this is what reveals trends, spikes and anomalies.



Unit economics

Cost per customer, per transaction, per active user. The measure of whether spend growth is healthy.



Near real-time data

Monthly reporting is too slow. Cost data must arrive with enough immediacy to respond to anomalies.

The goal

Move from *“IT is overspending”* to **“Team X used Y resources, costing \$Z, delivering A value.”**

THE VISIBILITY BLOCKERS

Five filters between us and a clear view



Inconsistent tagging

A significant portion of spend stays unallocated or misallocated.



Shared resources

Databases, Kubernetes clusters and networking resist clean attribution to one team.



Commitments & discounts

Reserved Instances and Savings Plans break attribution — list price $\neq$ price paid.



Multi-cloud estates

Each provider bills and reports differently; a unified view has to be constructed.



SaaS & marketplace spend

Invisible in cloud billing. Teams forget the subscriptions they signed.

“Different clouds report data differently, network traffic adds up fast, and Kubernetes or SaaS often hide who generated the spend.”

The foundation of all visibility



The challenge

“Tagging is the single most foundational habit for sustainable FinOps maturity. It is also the most frequently skipped.”



Define a global tag taxonomy

One master list, agreed across the business — aligned to business need, not just cloud ops.



Enforce tags at creation

Azure Policy, AWS Organizations tag policies, or Terraform. Retrofitting later is painful.



Start with showback

Show teams their spend, uncharged, for 4-6 weeks — gaps surface before money is on the line.



Monitor tag compliance

Dashboards that surface missing or incorrect metadata, tracked as a KPI.

Minimum viable tags

Environment

CostCenter

ApplicationID

Owner

Project

Attributing costs that belong to everyone

Shared databases, Kubernetes clusters and networking are the hardest costs to assign to a single owner.



Attribute-based allocation

Identify consumption at runtime and allocate on actual usage rather than an arbitrary key.



Define a splitting strategy

Pick one consistent method — proportional usage, team count, or a business metric — and publish it.



Start with showback

Report each team's share of shared cost without billing it. Builds awareness and surfaces disputes early.



Progress to chargeback

Bill teams on actual consumption once the usage patterns are understood and trusted.

“A thoughtful chargeback model often drives more intentional decisions and long-term discipline.”

SOLUTION 3 — COMMITMENTS & DISCOUNTS

Making the price we actually pay visible

Discounts obscure true attribution: the price shown on a resource is not the price the business paid.

60–80%

Target commitment coverage on stable, predictable workloads.



Separate discount from list price Show MSRP and the discounted price side by side, with the attributed saving alongside.



Attribute savings to the beneficiary If a team consumes the resources that earned the discount, that team should see the saving.



Track commitment coverage Know what share of steady-state spend sits under a commitment, and where the gaps are.



Monitor utilization An unused commitment is not a discount — it is prepaid waste.

A commitment nobody uses is the most expensive form of waste — it is locked in, and it is invisible on a list-price report.

SOLUTION 4 — MULTI-CLOUD VISIBILITY

One view across every provider

AWS Cost Explorer, Azure Cost Management and GCP Billing are capable — but each sees only its own silo.



Centralized platform

Ingest, normalize and enrich cost data from every provider into a single store.



Adopt the FOCUS standard

The FinOps Open Cost and Usage Specification normalizes cloud billing into a vendor-agnostic schema.



Unified dashboard

One place for analysis, reporting and optimization — across all clouds and vendors.

“FOCUS enables consistent reporting across vendors, so leadership can analyze the whole environment instead of looking at each cloud or vendor in a silo.”

Showback, then chargeback



Showback

The visibility
layer

Teams see their spend. No money changes hands.

Best for

- Early FinOps practice
- Tagging coverage of 70–85%
- Building awareness and trust



Chargeback

The real
consequence

Costs are billed back to business units and hit their P&L.

Best for

- Mature, trusted allocation
- Budget-owner accountability
- Driving behaviour change

Recommendation: start with showback to build tagging discipline and trust. Move to chargeback once allocation is accurate and finance is ready.

“Most teams skip this runway and pay for it later with a chargeback rollout that nobody trusts.”

What high-performing FinOps teams do

Drawn from the 2025 State of FinOps report — the practices that consistently precede durable savings.



Clean allocation

100% of spend allocated. No “unallocated” bucket left to argue about.



Daily visibility

Cost data refreshed daily, not monthly. Monthly is too slow to catch a regression.



Shared KPIs

One agreed set of metrics across finance and engineering, reviewed together.



Shift-left cost visibility

Cost checks during design and development — treated like performance and security.



Anomaly alerts

Real-time alerts routed to the owner of the spend, not to a shared mailbox.



Automated guardrails

Policy-as-code that prevents costly deployments before they happen.

PART TWO

Governance

Visibility tells you what happened. Governance decides what is allowed to happen — who owns the spend, what it may reach, and what stops it automatically.

Reporting is not a control. Every item on the right exists because someone found that out the expensive way.

01 Application ownership
A named owner for every workload

02 Budget controls
Thresholds that trigger before month-end

03 Rate limits & quotas
The only control fast enough to stop spend

04 Automated alerts
Routed to the person who can act

05 Shared responsibility
Who decides, who builds, who pays

Every workload has a named owner



Owner of record

A named person, not a distribution list and not “the platform team”. A group address means nobody answers the alert at 2am on a Sunday.



Owns the budget

Signs off the application's budget and explains the variance when it moves.



Answers the alerts

Named as the routing target for anomalies and threshold breaches on their workload.



Reviews monthly

Attends the monthly review with their own numbers, not a summary of everyone's.



Decommissions

Retires what is no longer used. Nothing else in the estate does this by itself.

Unowned is not unowned

Every resource without an owner already has one — the central budget. Make ownership a deployment requirement rather than a reporting field, and give orphaned resources a standing rule: no owner, no renewal.

Thresholds that fire before month-end

A budget nobody is warned about is a number in a spreadsheet, not a control.



Make it proportional, not flat

Day 10 of a 30-day month should be near a third of budget. At 60%, that is the alert. A flat “80% of budget” threshold fires on day 24 of a healthy month and day 8 of a catastrophic one — with equal urgency, and therefore equal indifference.

The only control fast enough to stop spend

How fast each signal moves

Billing data

24-48h behind

Utilization metrics

~5 minutes

Application telemetry

seconds

*“A cost alert cannot save you from a runaway job.
By the time actual cost moves, you have already
spent the money.”*



Per-deployment quotas

Cap tokens or requests per minute on each deployment.



Gateway throttling

Rate-limit per consumer key, so one client cannot drain the shared pool.



Autoscale ceilings

A maximum instance count on every scale set. Scaling is not a budget.



Non-prod shutdown

Scheduled stop outside working hours — the cheapest saving available.

Three tiers, three destinations

Sort alerts by what they can actually prevent — then route them accordingly.

Tier	Built on	Latency	Routes to
Protective	Tokens, utilization, error rates	minutes	On-call — page it
Budget	Billing data	24-48 hours	Team channel, weekly digest
Advisory	Billing plus forecast	daily	A ticket. Never a page.



Never page on a budget threshold

You are waking someone about a fact they cannot change.



A muted alert is worse than none

It leaves the false confidence that something is watching.



Alert on silence too

A stopped data feed reads exactly like a healthy month.

Who decides, who builds, who pays

FinOps fails when it is one team's job. These are the four parties and what each one owns.

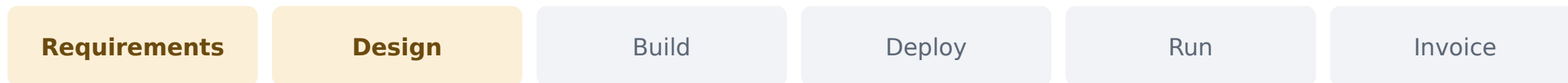
	Finance	FinOps / Platform	Engineering	Leadership
Budgets & allocation policy	Accountable			Contributes
Tagging & ownership standard	Contributes	Accountable		
Guardrails, quotas & alerts		Accountable	Contributes	
Responding to an alert		Contributes	Accountable	
Commitment & tooling spend	Contributes	Contributes		Accountable

Accountable — the decision stops here Contributes — does the work, does not decide

Shift left is a seat at the design table

Not a tool. A change in when FinOps shows up — and the earlier it is, the less of the bill is already decided.

Where FinOps belongs



Most of the lifetime cost is committed here.

Where FinOps usually arrives

WHAT FINOPS BRINGS TO THE ROOM

- **Two or three options, priced** The choice gets made with the number visible.
- **A cost target for the workload** Recorded beside latency and availability.
- **The allocation plan** Which tags, which cost centre, which unit.

WHAT FINOPS NEEDS TO LEARN FROM THE TEAM

- **What the project actually needs** Scale, growth, retention, and the real SLO.
- **What is temporary** An end date set now costs nothing to honour later.
- **Where the business value sits** Per customer, per order, per model call.

The FinOps Framework names this: Architecting & Workload Placement, and Onboarding Workloads — cost, usage and impact decision support during design and intake.

Six questions, asked before the build starts

None of them is a finance question. All of them decide the bill.



Shape of the load

Peak versus average, and how spiky? Sizing for a peak that lasts an hour a week is the most common overspend.



Data volume and retention

How much is stored, for how long, and how much leaves the region? Egress and retention rarely appear in the estimate.



What the SLO actually requires

Multi-region and multi-AZ are priced choices. Ask which one the business will genuinely pay for.



What is temporary

Non-production with no end date becomes permanent. An expiry set on day one is free; removing it later is a project.



For AI workloads

Tokens per request, cache hit rate, model tier, and whether provisioned throughput is justified by the traffic.



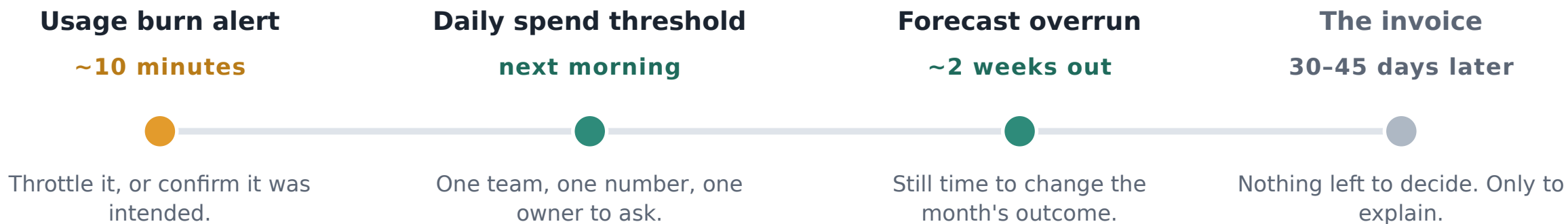
Owner and unit

Which cost centre carries it, and what unit we will divide the cost by once it is live.

The output is a number attached to the design: an estimate, and a cost target recorded beside the latency and availability targets. A cost diff on the pull request and policy as code keep it honest afterwards.

Nobody should learn it from the invoice

The same overrun, found at four different moments. Only the first three leave you anything to do about it.



What has to be true

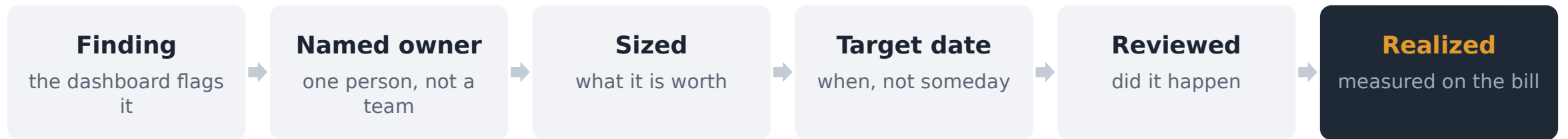
- ✓ Protective alerts run on usage data, not on billing data
- ✓ Every alert has a named owner to route to
- ✓ Budgets are phased, so drift shows up in week one
- ✓ Quotas cap the worst case while someone investigates

The point

The goal is not a smaller invoice. It is an invoice with nothing in it you have not already seen, explained and decided about.

Ship the insight, don't just publish it

A report tells someone a number. Only a change in what they do next reaches the bill.



A recommendation without an owner and a date is an opinion.

The difference that matters

A report is finished when the number is right.

A product is finished when someone behaves differently.

Build for the engineer who wants the three things worth fixing this sprint — the finance report improves as a by-product.

Measure adoption, not accuracy

- **Findings with an owner and a date**
The share that became work.
- **Realized against identified saving**
Only the first reaches an invoice.
- **Time from finding to owner**
Days, not weeks — and trending down.

The goal is a better decision, not a lower bill — the lower bill is what a better decision looks like a quarter later.

Where to start

01

Set the tagging standard

Agree a consistent taxonomy before investing in dashboards.

02

Start with the biggest spend

Attack the largest categories first. Don't chase perfect granularity everywhere.

03

Get finance and engineering talking

Bring both into the conversation early, not as an afterthought.

04

Set guardrails early

Owners, budget thresholds and quotas cost little to add — and stop the expensive surprises.

Tooling — match the footprint, not the ambition

Native

AWS Cost Explorer · Azure Cost Management · GCP Billing

Third-party

CloudHealth · Apptio · Vantage · nOps · Finout · CoreStack

Standard

FOCUS-compliant platforms for multi-cloud reporting

KEY TAKEAWAYS

Visibility is the foundation, not the goal

Visibility

enables accountability

Teams can only own what they can see.

Accountability

drives behaviour

Engineers make cost-conscious calls when they see the impact.

Behaviour

creates culture

Cost awareness becomes part of how we build, not a separate exercise.

Culture

sustains savings

Optimization becomes continuous rather than a one-time project.

“Organizations that invest in visibility first achieve more durable and targeted cost improvements than those that jump straight to cuts.”

Questions & discussion

FinOps: Visibility & Governance — seeing cloud spend, and controlling it

Key contacts

_____	· Title
_____	· Title

Resources

FinOps Foundation	·	finops.org
FOCUS standard	·	focus.finops.org

